

Algorytmy grafowe

Maria Linkowska 167906, Sara Rachmańczuk 167898

1. **Celem** zadania była implementacja dwóch metod sortowania topologicznego skierowanych grafów acyklicznych (DAG) – algorytmu Tarjana (opartego na przeszukiwaniu w głąb, DFS) oraz algorytmu Kahna (opartego na metodzie usuwania wierzchołków o zerowym stopniu wejściowym). Badania koncentrowały się na przeprowadzeniu analizy porównawczej czasów wykonania obu algorytmów w zależności od rozmiaru danych (liczby wierzchołków n), nasycenia grafu (gęstości s) oraz zastosowanej reprezentacji maszynowej grafu w pamięci.

2. **Badane struktury maszynowe grafu**

W ramach eksperymentów zaimplementowano generatory spójnych grafów DAG oraz trzy podstawowe reprezentacje grafów w pamięci:

- a. **Macierz sąsiedztwa (MS):** Reprezentacja dwuwymiarowa (tablica $n \times n$), w której komórka (i, j) przechowuje informację (np. wartość binarną) o istnieniu krawędzi skierowanej od wierzchołka i do wierzchołka j . Zaletą jest stały czas sprawdzania istnienia krawędzi $O(1)$, natomiast główną wadą jest stałe, wysokie zapotrzebowanie na pamięć $O(n^2)$ oraz złożoność czasowa przeszukiwania sąsiadów niezależna od faktycznej liczby krawędzi.
- b. **Lista następników (LN):** Reprezentacja wykorzystująca wektor (lub tablicę) wskaźników/list o rozmiarze n . Dla każdego wierzchołka i przechowywana jest lista sąsiadów, do których prowadzą bezpośrednio z niego krawędzie wychodzące. Struktura ta optymalizuje zużycie pamięci $O(n + m)$, gdzie m to liczba krawędzi, oraz pozwala na iterację wyłącznie po istniejących połączeniach.
- c. **Lista krawędzi (LK):** Najprostsza reprezentacja, polegająca na przechowywaniu jednowymiarowej, nieuporządkowanej struktury (wektora) przechowującej wszystkie krawędzie w postaci par wierzchołków (u, v) . Zajmuje pamięć $O(m)$, jednak weryfikacja sąsiedztwa danego wierzchołka wymaga zazwyczaj iteracji po całej liście, co znacząco rzutuje na złożoność algorytmów.

3. **Metodologia testów i pomiarów**

W celu przeprowadzenia rzetelnej analizy porównawczej, opracowano rygorystyczną procedurę pomiarową. Zaprojektowano dwa niezależne eksperymenty badające czas sortowania topologicznego w milisekundach [ms]:

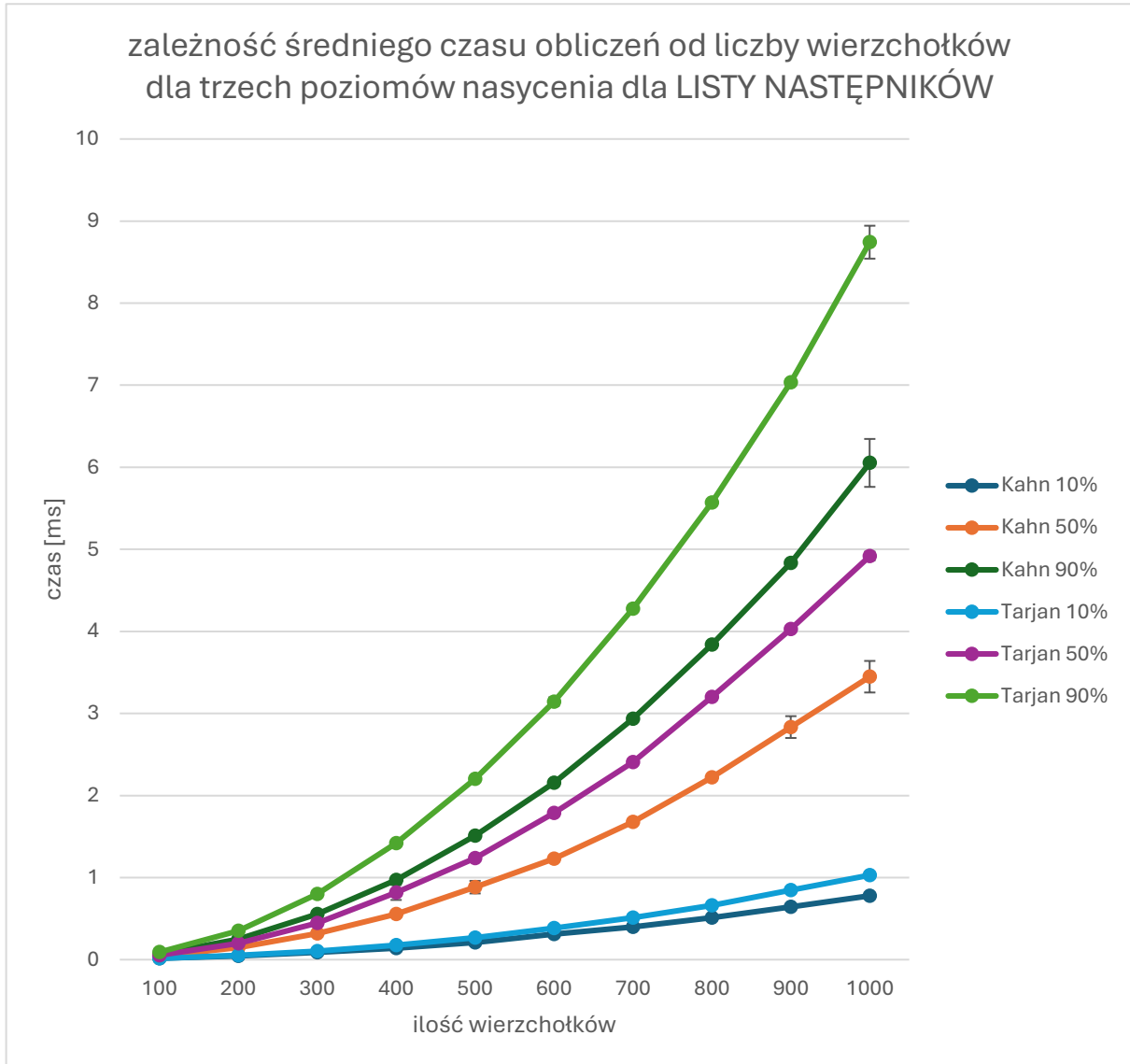
- a. **Eksperyment 1 (Liniowy wzrost n):** Badanie wpływu rosnącej liczby wierzchołków na czas obliczeń przy stałych poziomach nasycenia grafu. Wybrano zakres od $n=100$ do $n=1000$ (krok co 100), badając grafy rzadkie ($s=10\%$), średnio gęste ($s=50\%$) oraz bardzo gęste ($s=90\%$).
- b. **Eksperyment 2 (Liniowy wzrost s):** Badanie wpływu rosnącej gęstości grafu na czas obliczeń przy stałym rozmiarze danych. Ustalono stałą liczbę wierzchołków $n=500$, testując nasycenia rosnące liniowo od $s=10\%$ do $s=90\%$ (krok co 10%).

Dla każdego unikalnego punktu pomiarowego (kombinacja algorytmu, reprezentacji, n oraz s) wygenerowano 10 niezależnych instancji grafów losowych. Wynikiem punktowym zaprezentowanym na wykresach jest średnia arytmetyczna czasów tych iteracji. W celu weryfikacji stabilności pomiarowej algorytmów, dla każdego punktu wyliczono oraz naniesiono na wykresy odchylenie standardowe w postaci słupków błędów.

4. Analiza skalowalności algorytmów przy rosnącym rozmiarze danych (n)

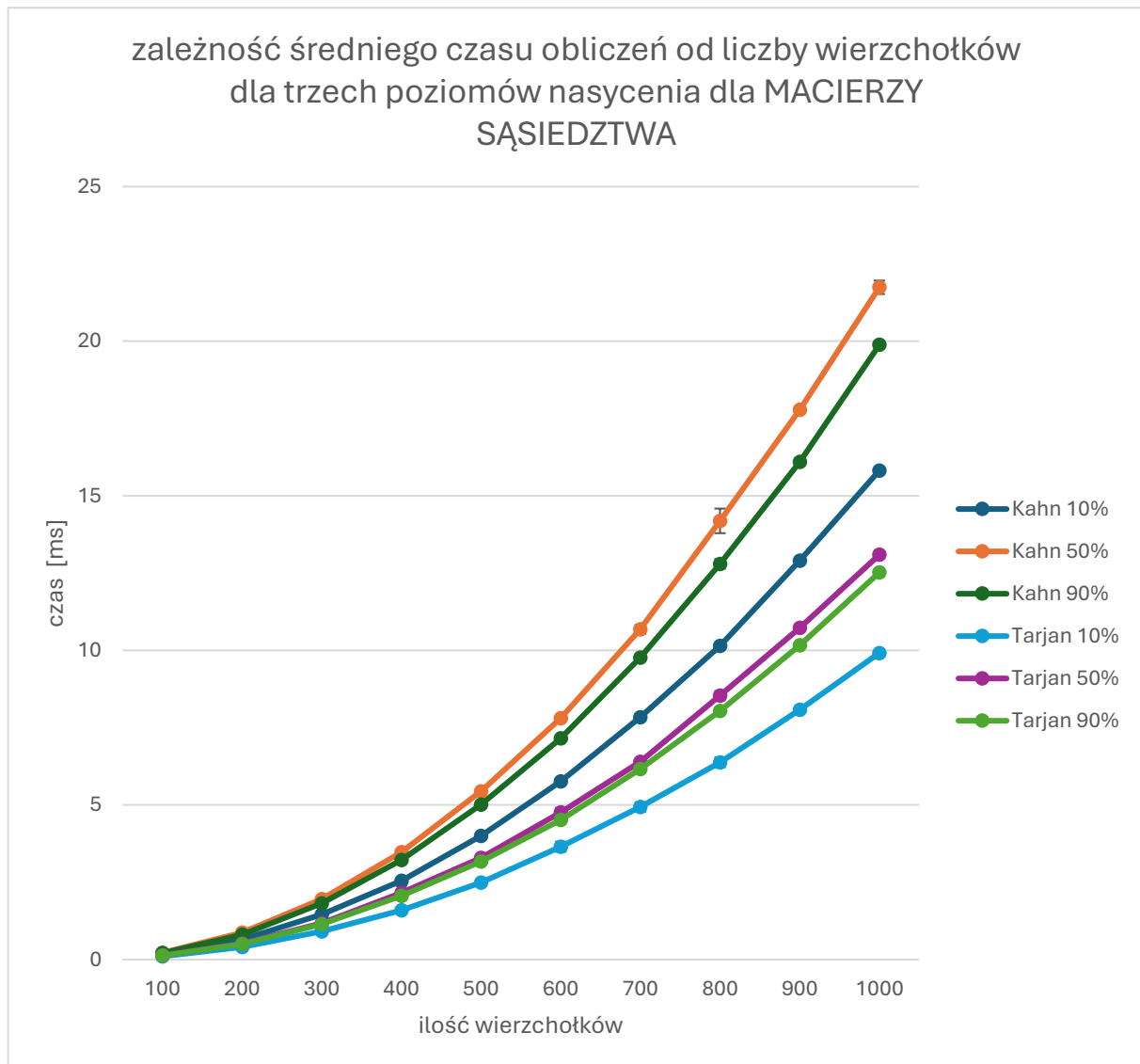
Pierwszy eksperyment skupiał się na zbadaniu, jak algorytmy radzą sobie ze stałym procentowym nasyceniem krawędzi, podczas gdy całkowity rozmiar grafu (liczba wierzchołków n) rośnie liniowo od 100 do 1000.

a. Macierz sąsiedztwa



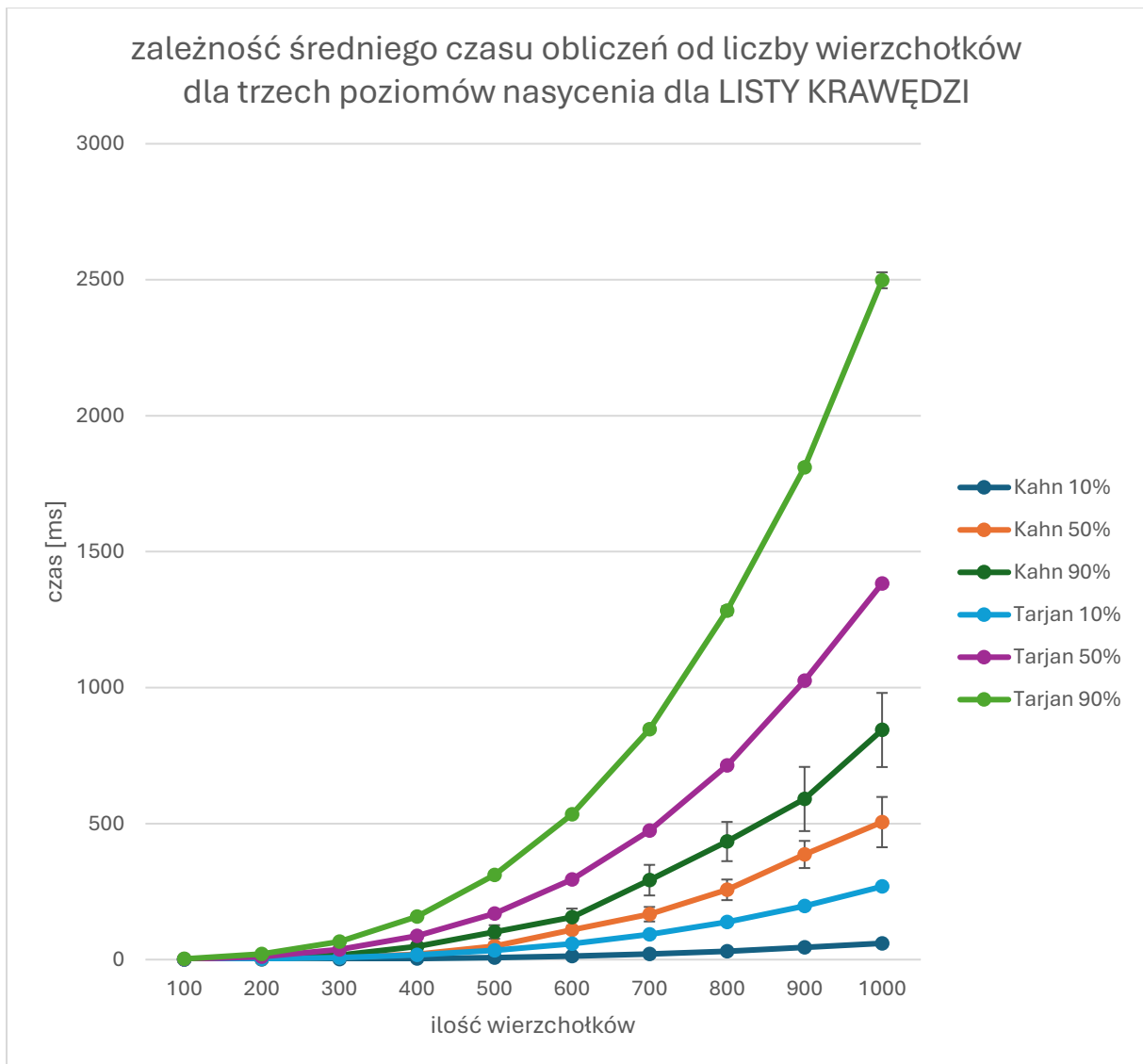
W przypadku macierzy sąsiedztwa zaobserwowano wyraźny, kwadratowy wzrost czasu obliczeń względem rosnącej liczby wierzchołków, co w pełni pokrywa się z przewidywaną złożonością teoretyczną rzędu $O(n^2)$ dla tej struktury. Wykres pokazuje ciekawe zjawisko wpływu gęstości grafu na czas wykonania, ponieważ oba algorytmy pracują nieznacznie szybciej dla grafów o bardzo wysokim nasyceniu (90%) niż dla grafów o nasyceniu średnim (50%). Wynika to ze specyfiki przeglądania macierzy – w gęstszych grafach warunki kończące pętle wyszukujące krawędzie są spełniane zauważalnie szybciej. Zestawienie obu metod jednoznacznie wskazuje, że algorytm Tarjana (oparty na przeszukiwaniu DFS) osiąga systematycznie lepsze czasy niż algorytm Kahna. Jest to podyktowane mniejszym narzutem operacji przygotowawczych przy przeszukiwaniu pamięciowej reprezentacji macierzowej.

b. Lista następników



Lista następników okazała się bezspornie najbardziej efektywną i skalowalną strukturą pamięciową spośród badanych. Nawet dla maksymalnego rozmiaru grafu rzędu $n = 1000$ wierzchołków, czasy wykonania algorytmów nie przekraczają progu 10 ms. Analiza wykresu ukazuje jednak istotną różnicę w zachowaniu tej struktury w porównaniu do macierzy sąsiedztwa: czas obliczeń rośnie tutaj proporcjonalnie do stopnia nasycenia grafu. Zjawisko to jest konsekwencją sposobu działania struktury o złożoności $O(n+m)$ – im wyższa liczba krawędzi (m), tym dłuższe są listy następników, przez które algorytm musi iterować. Lista następników wykazuje również wyjątkową precyzję uzyskanych wyników. Naniesione na wykres słupki błędów są minimalne, co dowodzi bardzo wysokiej stabilności czasowej i determinizmu działania algorytmów przy wykorzystaniu tej struktury danych.

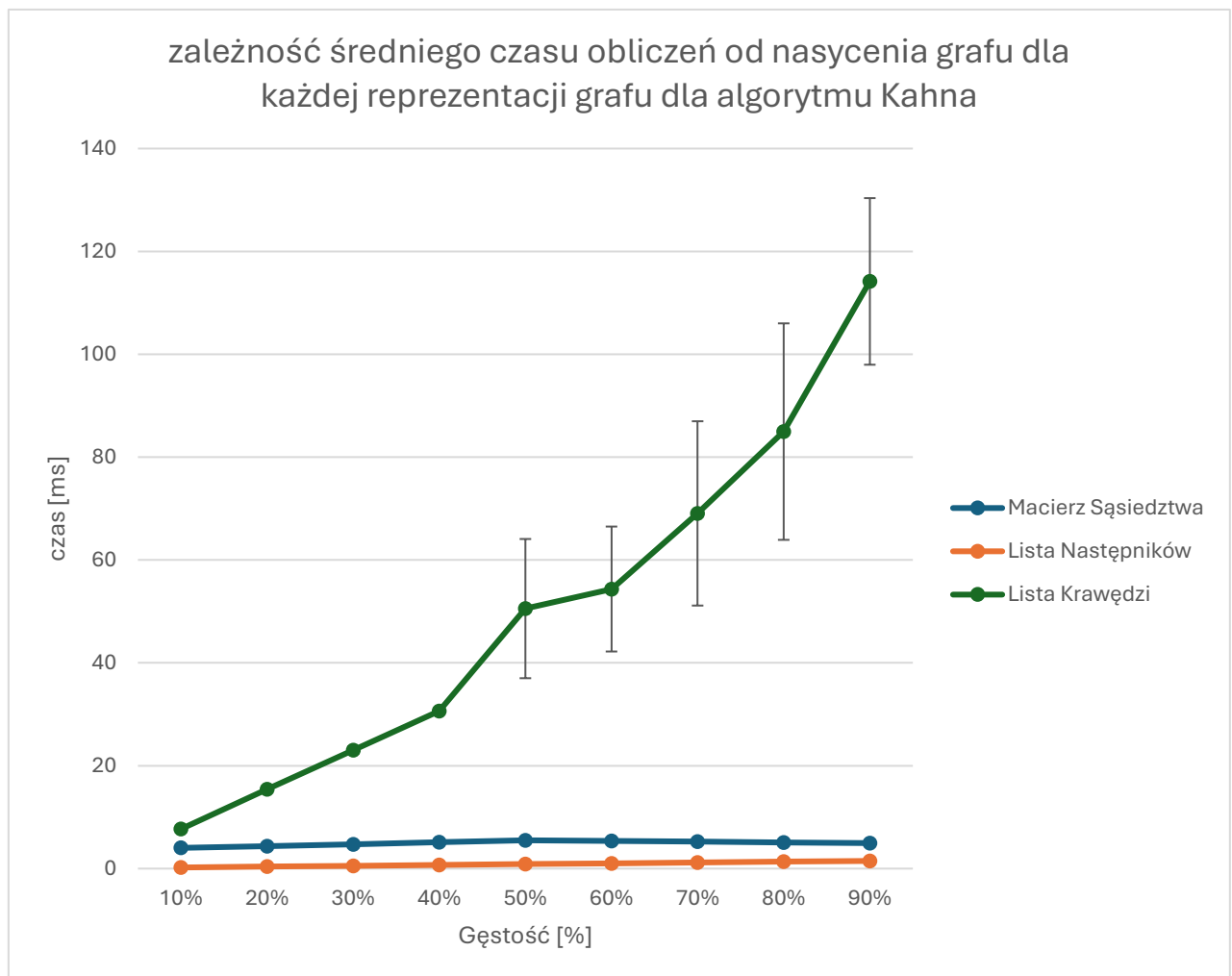
c. Lista krawędzi



Wykres dla listy krawędzi obrazuje drastyczny, nieliniowy wzrost czasu obliczeń, co czyni ją skrajnie niewydajną. Konieczność wielokrotnego, wyczerpującego przeszukiwania całej listy krawędzi w celu weryfikacji sąsiedztwa wierzchołków sprawia, że czasy wykonania są o dwa rzędy wielkości gorsze niż w przypadku zoptymalizowanej listy następników. W przeciwieństwie do poprzednich struktur, na liście krawędzi algorytm Kahna wykazuje znaczną przewagę nad algorytmem Tarjana. Średni czas wykonania algorytmu Kahna dla największych grafów oscyluje wokół 850 ms, podczas gdy dla algorytmu Tarjana wartości te osiągają krytyczny poziom blisko 2500 ms. Taka dysproporcja wynika z mechaniki algorytmu Kahna, który operuje na tej nieuporządkowanej strukturze w sposób bardziej usystematyzowany podczas początkowej aktualizacji stopni wejściowych.

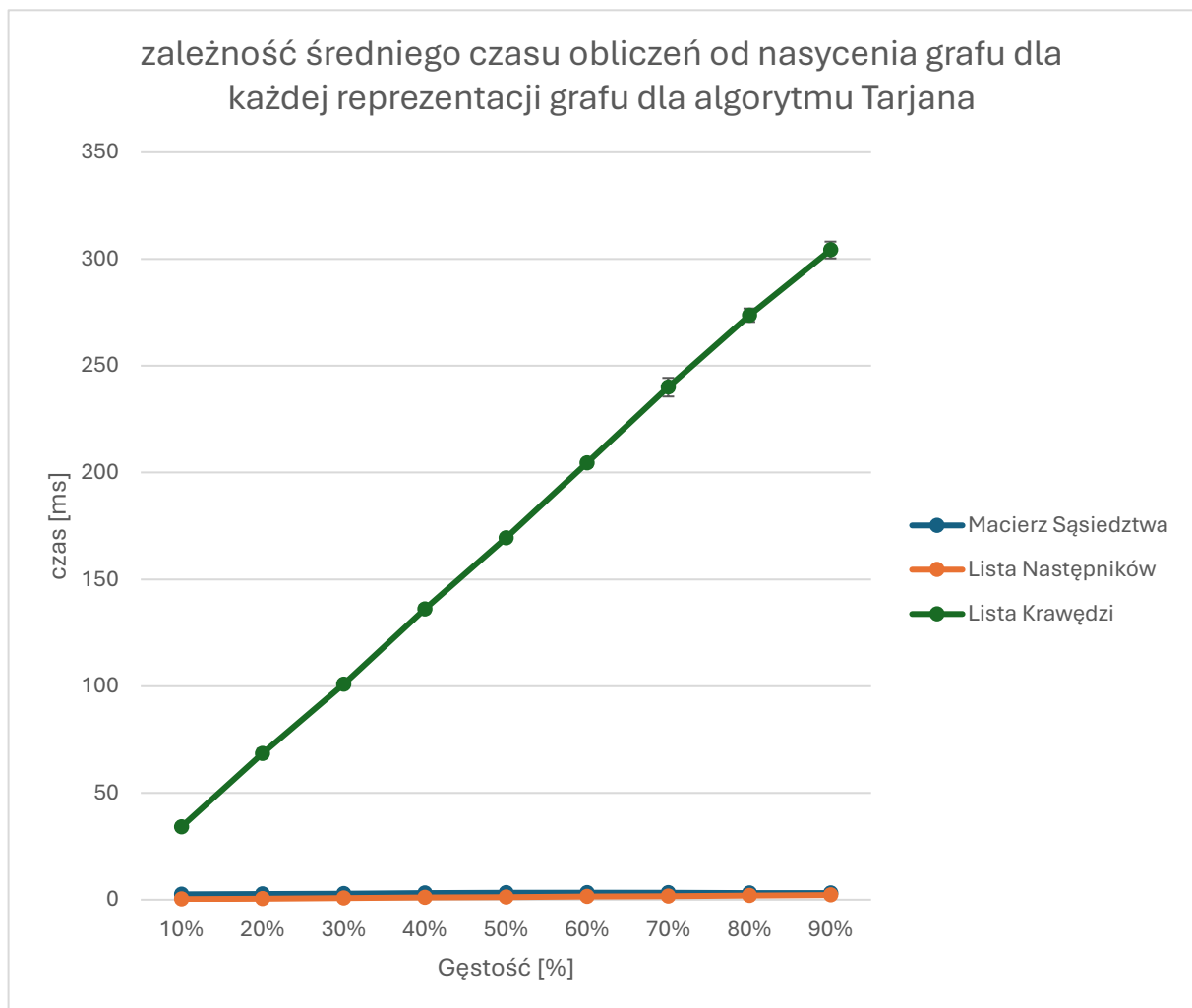
5. Analiza wyników - wpływ stopnia nasycenia (s) na efektywność reprezentacji maszynowych

a. Algorytm Kahna



- I. **Lista następników** – analiza wpływu gęstości grafu uwydatnia, że lista następników pozostaje rozwiązaniem bezkonkurencyjnym przy niskim nasyceniu krawędzi. Niemniej jednak, wraz ze zwiększaniem gęstości jej czas wykonania wykazuje tendencję rosnącą, sukcesywnie zbliżając się do wyników osiągniętych przez macierz sąsiedztwa.
- II. **Lista Krawędzi** – W drastycznym kontraście pozostaje lista krawędzi, dla której zaobserwowano stromy, liniowy wzrost czasu. Jest to bezpośrednim efektem faktu, że dla algorytmu Kahna koszt wyznaczania stopni wejściowych rośnie ściśle proporcjonalnie do całkowitej liczby krawędzi m w systemie.
- III. **Macierz Sąsiedztwa** – macierz sąsiedztwa charakteryzuje się imponującą odpornością na zmiany nasycenia. Czas jej przetwarzania utrzymuje się na niemal stałym poziomie około 5 ms w całym analizowanym spektrum gęstości, co potwierdza założenie, iż macierz wymusza wykonanie tej samej, stałej liczby sprawdzeń niezależnie od fizycznej obecności krawędzi.

b. Algorytm Tarjana



- I. **Lista Krawędzi** – w przypadku algorytmu opartego na metodzie przeszukiwania w głąb (DFS), uzyskany wykres bezsprzecznie dyskwalifikuje listę krawędzi jako poprawny wybór projektowy. Krzywa jej wydajności gwałtownie wzrasta wraz ze stopniem nasycenia, udowadniając, że identyfikacja sąsiadów w nieuporządkowanym wektorze krawędzi generuje krytyczne koszty obliczeniowe.
- II. **Lista Następników** – wykazuje znakomitą wydajność i znajduje się bardzo blisko osi OX, szczególnie przy grafach o niskim nasyceniu. Zauważalna jest tu jednak wyraźna tendencja rosnąca – wraz ze zwiększaniem gęstości grafu czas obliczeń liniowo się wydłuża. Wynika to z mechaniki działania algorytmu opartego na przeszukiwaniu w głąb (DFS), który dla każdego odwiedzanego wierzchołka zmuszony jest do iterowania po sukcesywnie wydłużających się listach sąsiadów.
- III. **Macierz sąsiedztwa** – wykres dla tej reprezentacji obrazuje wyjątkową stabilność i prezentuje krzywą niemal całkowicie płaską, o czasach działania bardzo zbliżonych do listy następników przy wyższych gęstościach (przy stałym $n = 500$ różnice są marginalne). Tak płaski przebieg krzywej to efekt stałego narzutu weryfikacji sąsiadów – niezależnie od tego, czy krawędzi jest mało czy dużo, algorytm Tarjana musi zawsze sprawdzić cały wiersz macierzy (wszystkie n kolumn) dla każdego badanego wierzchołka, co czyni tę strukturę wysoce uodporną na zmienne nasycenie.

6. Analiza wyników obu eksperymentów

- **Która reprezentacja maszynowa grafu jest najlepsza dla rzadkich grafów?**

Z wykorzystanych reprezentacji maszynowych najwyższą efektywność dla grafów rzadkich ($s = 10\%$) wykazuje lista następników. Wynika to z faktu, że algorytmy operujące na tej strukturze przeglądają wyłącznie istniejące krawędzie, co minimalizuje liczbę operacji procesora.

- **Która reprezentacja maszynowa grafu traci efektywność dla gęstych grafów?**

Największy spadek wydajności odnotowano dla listy krawędzi. Przy wzroście nasycenia do 90% czas wykonania algorytmu Tarjana dla $n = 1000$ wzrasta drastycznie do poziomu ok. 2500 ms, podczas gdy dla listy następników pozostaje poniżej 10 ms.

- **Czy macierz sąsiedztwa jest efektywna w algorytmie Tarjana (i dlaczego tak lub nie)?**

Macierz sąsiedztwa wykazuje stabilność przy dużych gęstościach, jednak nie jest optymalna dla algorytmu Tarjana. Powodem jest konieczność każdorazowego przeszukiwania całego wiersza macierzy (n elementów) w poszukiwaniu sąsiadów, co narzuca złożoność $O(n^2)$ niezależnie od liczby faktycznych krawędzi.

- **Czy lista następników jest korzystna dla algorytmu Kahna (i dlaczego tak lub nie)?**

Tak, jest to najbardziej korzystna struktura dla tego algorytmu. Pozwala na natychmiastowy dostęp do sąsiadów usuwanego wierzchołka i szybką aktualizację ich stopni wejściowych, co potwierdzają najniższe czasy we wszystkich próbach eksperymentu 1.

- **Jak koszt wyznaczania stopni wejściowych wpływa na algorytm Kahna?**

Koszt ten stanowi istotny narzut początkowy (wymaga wykonania operacji przygotowawczych przed właściwym sortowaniem). W reprezentacjach macierzy sąsiedztwa i listy krawędzi wymaga on pełnego przejścia przez strukturę danych (odpowiednio n^2 komórek lub m krawędzi), co jest szczególnie widoczne w eksperymencie 2, gdzie czas Kahna dla listy krawędzi rośnie liniowo wraz z gęstością grafu.

- **Czy różnice między reprezentacjami rosną wraz z liczbą wierzchołków lub gęstością grafu?**

Tak, dysproporcje rosną nieliniowo wraz ze wzrostem n . O ile dla $n=100$ różnice są rzędu milisekund, o tyle dla $n=1000$ różnica między listą następników a listą krawędzi jest już kilkusetkrotna. Z kolei wraz ze wzrostem gęstości grafu (Eksperyment 2) różnice również się pogłębiają – przy zwiększaniu nasycenia czas działania dla listy krawędzi rośnie bardzo szybko, podczas gdy dla macierzy sąsiedztwa pozostaje on niemal stały, co tworzy coraz większą przepaść między tymi reprezentacjami.

7. Analiza złożoności obliczeniowej

Poniższa tabela przedstawia porównania teoretycznych złożoności czasowych z wynikami zaobserwowanymi w eksperymentach:

Reprezentacja	Algorytm Tarjana (DFS)	Algorytm Kahna	Zgodność z wynikami
Macierz sąsiedztwa	$O(n^2)$	$O(n^2)$	Wysoka zgodność – wykresy mają kształt paraboli
Lista Następników	$O(n+m)$	$O(n+m)$	Wysoka zgodność – najniższe czasy, bliskie liniowości
Lista Krawędzi	$O(n*m)$	$O(n*m)$	Wysoka zgodność – gwałtowny wzrost czasu przy dużych m

W reprezentacji listy krawędzi algorytmy muszą przeszukiwać listę wszystkich krawędzi w poszukiwaniu sąsiadów każdego wierzchołka, co tłumaczy uzyskaną złożoność rzędu $O(n*m)$ i najgorsze wyniki czasowe.

Analiza wykazała **całkowity brak rozbieżności** pomiędzy przewidywaniami teoretycznymi a wynikami eksperymentalnymi – uzyskane krzywe na wykresach idealnie odzwierciedlają szacowane klasy złożoności.

8. Wnioski końcowe

- **Dominacja Listy Następników** – jest to najbardziej uniwersalna i efektywna reprezentacja dla obu badanych algorytmów, zapewniająca najlepszą skalowalność.
- **Niewydajność listy krawędzi** – reprezentacja ta stanowi klasyczny, ale bardzo niekorzystny przykład **kompromisu pamięć vs czas**. Mimo ogromnych oszczędności pamięciowych ($O(m)$) w bardzo rzadkich grafach, koszt czasowy operacji ($O(n * m)$) jest absolutnie nieakceptowalny dla sortowania topologicznego w większych zbiorach danych.
- **Wybór algorytmu** – algorytm Tarjana (DFS) w większości przypadków osiągał nieco lepsze czasy od algorytmu Kahna, co wynika z braku konieczności wstępnego wyznaczania stopni wejściowych wszystkich wierzchołków.
- **Rekomendacje** – w praktycznych zastosowaniach inżynierskich, gdzie grafy są zazwyczaj rzadkie, należy stosować listę następników. Macierz sąsiedztwa może być rozważana wyłącznie przy ekstremalnie gęstych grafach. Występuje tu jednak inny **kompromis pamięć vs czas**: w zamian za stały czas weryfikacji krawędzi $O(1)$, musimy zaakceptować ogromne zapotrzebowanie na pamięć rzędu $O(n^2)$, co przy grafach o kilkudziesięciu tysiącach wierzchołków może przepełnić pamięć operacyjną.

9. Tabele wyników

Eksperyment 1:

N	Gestosc	Kahn_MS_sr	Kahn_MS_odch	Tarjan_MS_sr	Tarjan_MS_odch
100	0,1	0,16762	0,002102	0,10376	0,000738
200	0,1	0,64893	0,004356	0,40657	0,007099
300	0,1	1,45942	0,014697	0,91286	0,013623
400	0,1	2,54605	0,033051	1,59553	0,017766
500	0,1	3,99996	0,016938	2,49212	0,010402
600	0,1	5,76796	0,026198	3,65167	0,161352
700	0,1	7,83589	0,054756	4,93576	0,150676
800	0,1	10,1489	0,042385	6,37642	0,15734
900	0,1	12,9015	0,075297	8,08508	0,069183
1000	0,1	15,8158	0,073513	9,91807	0,076204

N	Gestosc	Kahn_MS_sr	Kahn_MS_odch	Tarjan_MS_sr	Tarjan_MS_odch
100	0,5	0,22156	0,001593	0,13386	0,002083
200	0,5	0,87307	0,007791	0,52286	0,004855
300	0,5	1,95621	0,012391	1,17702	0,003769
400	0,5	3,47435	0,049177	2,15678	0,220173
500	0,5	5,44336	0,064044	3,29399	0,075167
600	0,5	7,80925	0,04082	4,7648	0,144146
700	0,5	10,6789	0,147659	6,40162	0,051846
800	0,5	14,1888	0,398011	8,53609	0,148673
900	0,5	17,781	0,045393	10,7295	0,065111
1000	0,5	21,7432	0,217015	13,0937	0,088485

N	Gestosc	Kahn_MS_sr	Kahn_MS_odch	Tarjan_MS_sr	Tarjan_MS_odch
100	0,9	0,20794	0,001165	0,1297	0,00065
200	0,9	0,80975	0,00892	0,50621	0,00239
300	0,9	1,81525	0,015545	1,14799	0,023871
400	0,9	3,21781	0,013964	2,05971	0,053042
500	0,9	5,01036	0,020975	3,17099	0,014354
600	0,9	7,158	0,030139	4,51836	0,028598
700	0,9	9,7636	0,075646	6,16066	0,028167
800	0,9	12,7973	0,154461	8,0452	0,077576
900	0,9	16,1026	0,079462	10,1643	0,056519
1000	0,9	19,8832	0,120582	12,5192	0,111083

N	Gestosc	Kahn_LN_sr	Kahn_LN_odch	Tarjan_LN_sr	Tarjan_LN_odch
100	0,1	0,01709	0,000554	0,01559	0,000226
200	0,1	0,04525	0,000361	0,05055	0,001873
300	0,1	0,08874	0,004869	0,10442	0,001849
400	0,1	0,13943	0,002583	0,1763	0,002818
500	0,1	0,20963	0,004809	0,26885	0,002143
600	0,1	0,30941	0,00428	0,38283	0,005114
700	0,1	0,39874	0,009389	0,51312	0,007622
800	0,1	0,51161	0,006118	0,6625	0,016672
900	0,1	0,64225	0,008579	0,84732	0,018207
1000	0,1	0,77798	0,002389	1,0293	0,016901

N	Gestosc	Kahn_LN_sr	Kahn_LN_odch	Tarjan_LN_sr	Tarjan_LN_odch
100	0,5	0,04214	0,001473	0,05383	0,003481
200	0,5	0,14784	0,004307	0,19861	0,000688
300	0,5	0,31922	0,005891	0,44669	0,004813
400	0,5	0,5535	0,007798	0,81862	0,091333
500	0,5	0,88306	0,076557	1,23628	0,015621
600	0,5	1,23042	0,0148	1,78669	0,024914
700	0,5	1,67768	0,027898	2,40599	0,018448
800	0,5	2,22064	0,014278	3,20038	0,029982
900	0,5	2,83411	0,132778	4,02844	0,01958
1000	0,5	3,44762	0,192155	4,91723	0,053742

N	Gestosc	Kahn_LN_sr	Kahn_LN_odch	Tarjan_LN_sr	Tarjan_LN_odch
100	0,9	0,06836	0,00145	0,09086	0,000403
200	0,9	0,25082	0,003983	0,35127	0,002614
300	0,9	0,55407	0,011294	0,79978	0,014695
400	0,9	0,96991	0,003722	1,41866	0,015025
500	0,9	1,5117	0,014115	2,2014	0,013267
600	0,9	2,15468	0,021967	3,1436	0,052489
700	0,9	2,93417	0,012282	4,27528	0,018448
800	0,9	3,83702	0,021914	5,57083	0,033203
900	0,9	4,83286	0,034412	7,03614	0,035009
1000	0,9	6,05284	0,292191	8,74265	0,20054

N	Gestosc	Kahn_LK_sr	Kahn_LK_odch	Tarjan_LK_sr	Tarjan_LK_odch
100	0,1	0,07577	0,001161	0,27933	0,006483
200	0,1	0,52056	0,004395	2,26925	0,288255
300	0,1	1,69323	0,007836	7,36936	0,080595
400	0,1	3,91619	0,026067	17,2644	0,187576
500	0,1	7,62166	0,019023	33,7896	0,072548
600	0,1	13,1033	0,042637	58,1978	0,175034
700	0,1	20,7319	0,066227	92,891	0,476296
800	0,1	30,6724	0,135939	137,7	1,47834
900	0,1	45,177	3,79609	197,361	1,31483
1000	0,1	59,7645	0,189955	268,672	1,52006

N	Gestosc	Kahn_LK_sr	Kahn_LK_odch	Tarjan_LK_sr	Tarjan_LK_odch
100	0,5	0,34624	0,004967	1,37685	0,039739
200	0,5	2,51946	0,007713	10,7598	0,025619
300	0,5	8,37536	0,032019	36,675	0,213075
400	0,5	19,6865	0,501548	86,6306	1,21516
500	0,5	49,242	12,9409	169,688	2,02587
600	0,5	109,741	13,8885	294,346	2,55926
700	0,5	166,778	26,7633	474,493	4,38659
800	0,5	256,523	37,8756	714,107	11,0169
900	0,5	386,522	49,9344	1025,24	10,724
1000	0,5	505,519	92,4923	1382,93	12,3745

N	Gestosc	Kahn_LK_sr	Kahn_LK_odch	Tarjan_LK_sr	Tarjan_LK_odch
100	0,9	0,61941	0,004918	2,49854	0,036331
200	0,9	4,56667	0,031837	20,0246	1,86333
300	0,9	15,7029	1,28288	66,0879	0,457291
400	0,9	48,0134	9,11126	157,813	1,9546
500	0,9	101,596	24,8819	311,306	5,14614
600	0,9	155,528	32,0178	534,36	6,60189
700	0,9	292,086	56,2524	847,52	13,2052
800	0,9	434,098	72,1406	1282,88	16,7623
900	0,9	590,516	118,027	1810,12	14,253
1000	0,9	844,292	136,302	2497,97	29,3954

Eksperyment 2:

N	Gestosc	Kahn_MS_sr	Kahn_MS_odch	Kahn_LN_sr	Kahn_LN_odch	Kahn_LK_sr	Kahn_LK_odch
500	0,1	4,03132	0,039874	0,21554	0,009233	7,68898	0,104966
500	0,2	4,34519	0,022474	0,37872	0,009359	15,4302	0,512105
500	0,3	4,70163	0,022617	0,53574	0,008938	23,0194	0,69917
500	0,4	5,10769	0,027252	0,70075	0,012083	30,652	0,378715
500	0,5	5,49804	0,229924	0,89807	0,135131	50,5505	13,5327
500	0,6	5,38088	0,018881	1,01432	0,007973	54,3587	12,1472
500	0,7	5,25092	0,0215	1,18318	0,012068	69,059	17,9192
500	0,8	5,08065	0,027765	1,3367	0,020119	84,9632	21,0476
500	0,9	4,97146	0,033897	1,48392	0,0044	114,166	16,1932

N	Gestosc	Tarjan_MS_sr	Tarjan_MS_odch	Tarjan_LN_sr	Tarjan_LN_odch	Tarjan_LK_sr	Tarjan_LK_odch
500	0,1	2,52158	0,024109	0,27325	0,007151	34,0541	0,269779
500	0,2	2,69441	0,019202	0,51546	0,011966	68,4883	2,19295
500	0,3	2,88001	0,012825	0,75221	0,014607	100,879	0,547101
500	0,4	3,17175	0,134255	1,00234	0,015275	136,131	1,94636
500	0,5	3,27276	0,018406	1,23657	0,020496	169,419	0,311003
500	0,6	3,28725	0,021253	1,4731	0,015569	204,438	1,85283
500	0,7	3,23649	0,015007	1,70827	0,015202	239,983	4,36936
500	0,8	3,15702	0,020285	1,92589	0,017814	273,704	3,13372
500	0,9	3,13638	0,034004	2,18004	0,028502	304,239	3,91453

Algorytmy zakodowano w języku C++.

Sprzęt, na którym zostały wykonane pomiary:

Procesor: AMD Ryzen 7 3700X

RAM: 16GB DDR4 3200MHz

Karta Graficzna: NVIDIA GeForce RTX 2060

System operacyjny:

Windows 10